

Unit 1 – Experiment 4, 5, 6

CRUD Application Notes by Ashish Sir Sunstone

Beginner-friendly notes on database architectures (SQL vs NoSQL), pagination & sorting for large datasets, and secure file uploads. All code snippets are short and commented.

1. What is CRUD?

C – Create

Insert new records. `INSERT INTO users (name) VALUES ('Ravi');`

R – Read

Fetch records. `SELECT * FROM users WHERE id = 1;`

U – Update

Modify records. `UPDATE users SET name='Ravi K' WHERE id=1;`

D – Delete

Remove records. `DELETE FROM users WHERE id = 1;`

💡 CRUD = the four basic operations every data-driven app needs.

2. Database Architectures: Relational vs NoSQL

♦ Relational Databases (SQL)

- Store data in tables with rows and columns.
- Use SQL for queries. Strict schema.
- Examples: MySQL, PostgreSQL, SQLite.

♦ NoSQL Databases

- Store data as documents, key-value, or graphs.
- Flexible schema, horizontally scalable.
- Examples: MongoDB, Redis, Cassandra.

Feature	Relational (SQL)	NoSQL
Schema	Fixed (tables)	Flexible (documents)
Query Language	SQL	JSON-like / custom
Scaling	Vertical (bigger server)	Horizontal (more servers)
Best For	Complex joins, transactions	Big data, real-time apps
Speed	Fast for structured queries	Faster for huge unstructured data

```
// SQL (MySQL) - insert a user
$pdo->prepare("INSERT INTO users (name, email) VALUES (?, ?)")
->execute(['Ravi', 'ravi@mail.com']);

// NoSQL (MongoDB) - insert a user
$collection->insertOne(['name' => 'Ravi', 'email' => 'ravi@mail.com']);
// No fixed columns, can add extra fields anytime
```

🧠 Simple rule: Use SQL when data has clear relationships. Use NoSQL when data is huge, changes shape, or needs fast scaling.

3. Pagination (for Large Datasets)

Instead of loading 10,000 rows at once, we load only a few per page. Faster and friendlier.

📄 PAGINATION FLOW

```
[User requests page = 2, limit = 10]
↓
[Calculate offset = (page - 1) * limit = 10]
↓
[SQL: SELECT * FROM users LIMIT 10 OFFSET 10]
↓
[Show 10 rows + Prev/Next links]
```

```
// Pagination in PHP + MySQL
$page = $_GET['page'] ?? 1; // current page
$limit = 10; // rows per page
$offset = ($page - 1) * $limit;

// Fetch only needed rows
$stmt = $pdo->prepare("SELECT * FROM users LIMIT ? OFFSET ?");
$stmt->bindValue(1, $limit, PDO::PARAM_INT);
$stmt->bindValue(2, $offset, PDO::PARAM_INT);
$stmt->execute();
$rows = $stmt->fetchAll();

// Total rows (for page count)
$total = $pdo->query("SELECT COUNT(*) FROM users")->fetchColumn();
$pages = ceil($total / $limit);
```

♦ Sorting

Allow user to sort by a column (name, date, price) using `ORDER BY`.

```
// Sorting - whitelist columns to prevent SQL injection
$allowed = ['id', 'name', 'email'];
$sort = in_array($_GET['sort'] ?? '', $allowed) ? $_GET['sort'] : 'id';
$order = ($_GET['order'] ?? 'asc') === 'desc' ? 'DESC' : 'ASC';

$sql = "SELECT * FROM users ORDER BY $sort $order LIMIT ? OFFSET ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([$limit, $offset]);
```

⚠️ Never trust user input for column names. Always whitelist allowed columns.

4. File Upload with Validation & Security

Users upload files (images, PDFs). Must validate type, size, and store safely.

📁 FILE UPLOAD FLOW

```
[User selects file → submits form]
↓
[Check: size, extension, MIME type]
↓
[Generate safe random filename]
↓
[Move file to /uploads folder OUTSIDE web root]
↓
[Save filename in database]
```

♦ HTML Form

```
<form method="POST" enctype="multipart/form-data">
  <input type="file" name="photo">
  <button>Upload</button>
</form>
```

♦ PHP Upload Handler (Secure)

```
// Allowed file types and max size (2 MB)
$allowed = ['jpg', 'jpeg', 'png', 'pdf'];
$maxSize = 2 * 1024 * 1024;

$file = $_FILES['photo'];
$ext = strtolower(pathinfo($file['name'], PATHINFO_EXTENSION));

// 1. Check extension
if (!in_array($ext, $allowed)) {
    die("File type not allowed.");
}

// 2. Check size
if ($file['size'] > $maxSize) {
    die("File too large (max 2 MB).");
}

// 3. Check real MIME type (not just extension)
$mime = mime_content_type($file['tmp_name']);
$safeMimes = ['image/jpeg', 'image/png', 'application/pdf'];
if (!in_array($mime, $safeMimes)) {
    die("Invalid file content.");
}

// 4. Generate random filename (prevents overwrite & path tricks)
$newName = bin2hex(random_bytes(16)) . '.' . $ext;
$dest = __DIR__ . '/uploads/' . $newName;

// 5. Move file
if (move_uploaded_file($file['tmp_name'], $dest)) {
    // Save $newName in database
    echo "Uploaded successfully!";
} else {
    echo "Upload failed.";
}
```

🛡️ Security Rules

- Never trust the file extension alone — check MIME type.
- Rename files with random names.
- Store uploads *outside* the web root if possible.
- Disable script execution in the upload folder.
- Limit file size to prevent DoS.

✅ Good Practices

- Validate on both client and server.
- Use `move_uploaded_file()` only.
- Keep a whitelist of extensions.
- Log uploads for auditing.
- Serve files through PHP (not directly) if private.

5. How It All Fits – A Simple CRUD App

1. Database: Choose SQL (MySQL) for structured data, NoSQL (MongoDB) for flexible data.
2. Create: Form → validate → insert into DB (with file upload if needed).
3. Read: Use pagination (`LIMIT / OFFSET`) + sorting (`ORDER BY`).
4. Update: Fetch record → edit form → update DB.
5. Delete: Confirm → delete row + remove associated file.

🧠 Key takeaway: SQL for relationships, NoSQL for scale. Paginate large lists. Always validate and rename uploaded files — never trust users.